Time Series Anomaly Detection using LSTM Autoencoders with PyTorch in Python

22.03.2020 — Deep Learning, PyTorch, Machine Learning, Neural Network, Autoencoder, Time Series, Python — 5 min read

SHARE

> *TL;DR Use real-world Electrocardiogram (ECG) data to detect anomalies in a patient heartbeat. We'll build an LSTM Autoencoder, train it on a set of normal heartbeats and classify unseen examples as normal or anomalies*

In this tutorial, you'll learn how to detect anomalies in Time Series data using an LSTM Autoencoder. You're going to use real-world ECG data from a single patient with heart disease to detect abnormal hearbeats.

- Run the complete notebook in your browser (Google Colab)
- Read the Getting Things Done with Pytorch book

By the end of this tutorial, you'll learn how to:

- Prepare a dataset for Anomaly Detection from Time Series Data
- Build an LSTM Autoencoder with PyTorch

- Train and evaluate your model
- Choose a threshold for anomaly detection
- Classify unseen examples as normal or anomaly

Data

The dataset contains 5,000 Time Series examples (obtained with ECG) with 140 timesteps. Each sequence corresponds to a single heartbeat from a single patient with congestive heart failure.

> *An electrocardiogram (ECG or EKG) is a test that checks how your heart is functioning by measuring the electrical activity of the heart. With each heart beat, an electrical impulse (or wave) travels through your heart. This wave causes the muscle to squeeze and pump blood from the heart. Source*

We have 5 types of hearbeats (classes):

- Normal (N)
- R-on-T Premature Ventricular Contraction (R-on-T PVC)
- Premature Ventricular Contraction (PVC)
- Supra-ventricular Premature or Ectopic Beat (SP or EB)
- Unclassified Beat (UB).

> *Assuming a healthy heart and a typical rate of 70 to 75 beats per minute, each cardiac cycle, or heartbeat, takes about 0.8 seconds to complete the cycle. Frequency: 60–100 per minute (Humans) Duration: 0.6–1 second (Humans) Source*

The dataset is available on my Google Drive. Let's get it:

```bash
!gdown --id 16MIleqoIr1vYxlGk4GKnGmrsCPuWkkpT
```

```bash
!unzip -qq ECG5000.zip
```

```python
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

The data comes in multiple formats. We'll load the `arff` files into Pandas data frames:

```python
with open('ECG5000_TRAIN.arff') as f:
  train = a2p.load(f)

with open('ECG5000_TEST.arff') as f:
  test = a2p.load(f)
```

We'll combine the training and test data into a single data frame. This will give us more data to train our Autoencoder. We'll also shuffle it:

```python
df = train.append(test)
df = df.sample(frac=1.0)
df.shape
```

```
(5000, 141)
```

We have 5,000 examples. Each row represents a single heartbeat record. Let's name the possible classes:

```python
CLASS_NORMAL = 1

```

```
class_names = ['Normal','R on T','PVC','SP','UB']
```

Next, we'll rename the last column to `target`, so its easier to reference it:

PYTHON

```
new_columns = list(df.columns)
new_columns[-1] = 'target'
df.columns = new_columns
```

Exploratory Data Analysis

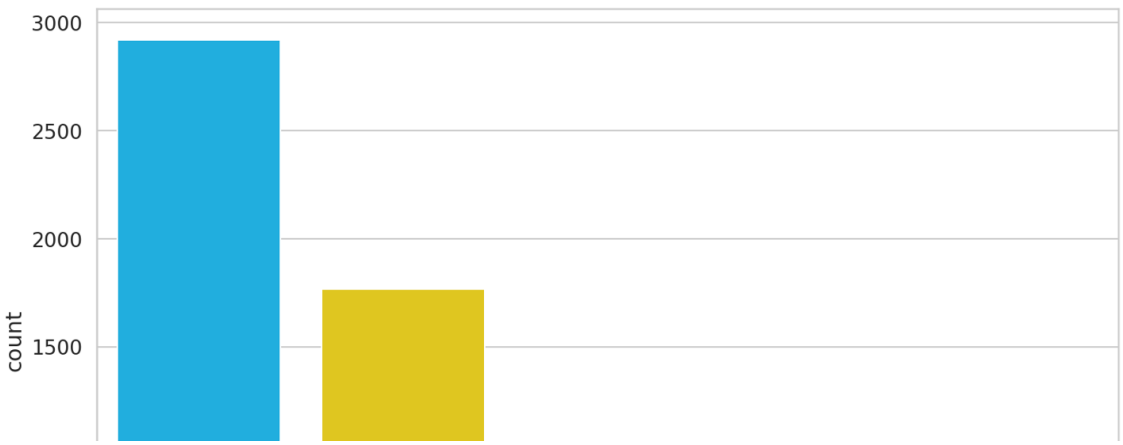
Let's check how many examples for each heartbeat class do we have:

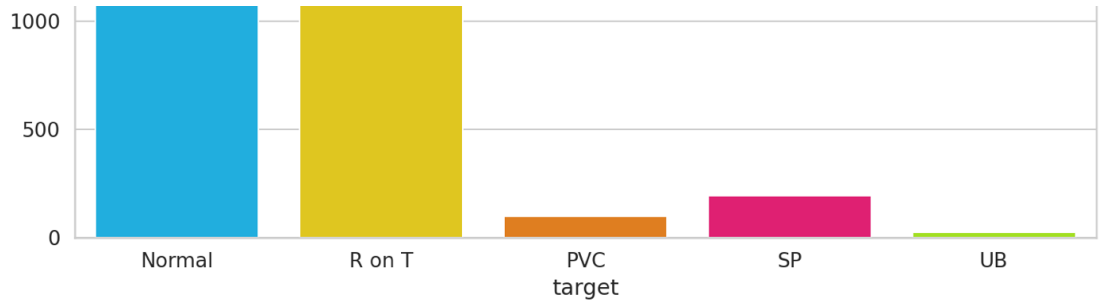
PYTHON

```
df.target.value_counts()
```

```
1    2919
2    1767
4     194
3      96
5      24
Name: target, dtype: int64
```

Let's plot the results:

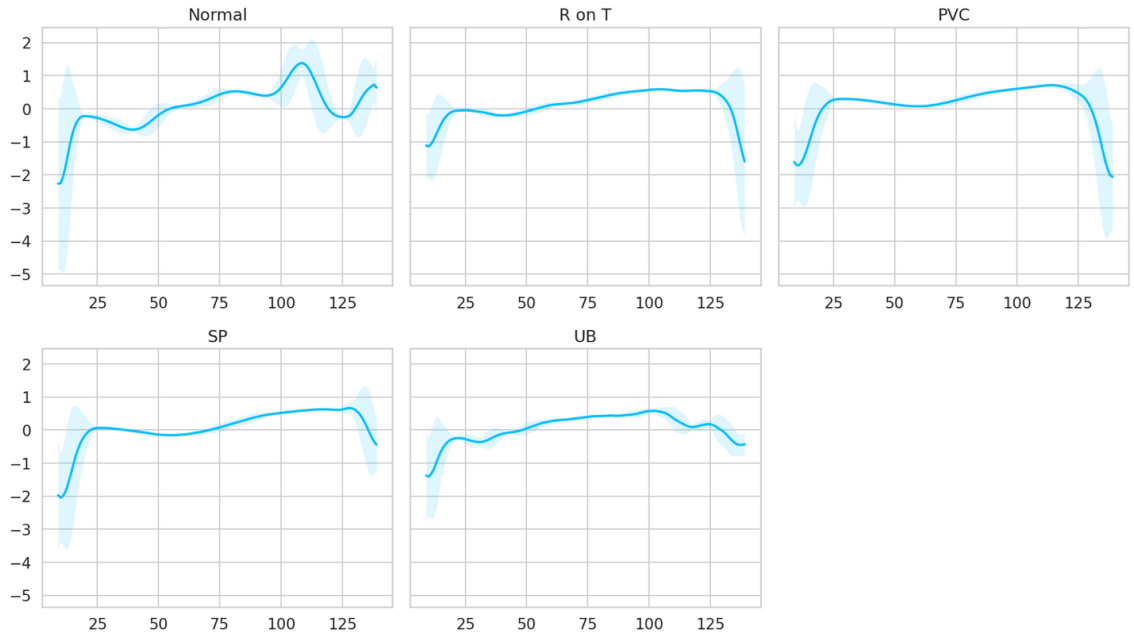




png

The normal class, has by far, the most examples. This is great because we'll use it to train our model.

Let's have a look at an averaged (smoothed out with one standard deviation on top and bottom of it) Time Series for each class:



png

It is very good that the normal class has a distinctly different pattern than all other classes. Maybe our model will be able to detect anomalies?

LSTM Autoencoder

The Autoencoder's job is to get some input data, pass it through the model, and

obtain a reconstruction of the input. The reconstruction should match the input as much as possible. The trick is to use a small number of parameters, so your model learns a compressed representation of the data.

In a sense, Autoencoders try to learn only the most important features (compressed version) of the data. Here, we'll have a look at how to feed Time Series data to an Autoencoder. We'll use a couple of LSTM layers (hence the LSTM Autoencoder) to capture the temporal dependencies of the data.

To classify a sequence as normal or an anomaly, we'll pick a threshold above which a heartbeat is considered abnormal.

Reconstruction Loss

When training an Autoencoder, the objective is to reconstruct the input as best as possible. This is done by minimizing a loss function (just like in supervised learning). This function is known as *reconstruction loss*. Cross-entropy loss and Mean squared error are common examples.

Anomaly Detection in ECG Data

We'll use normal heartbeats as training data for our model and record the *reconstruction loss*. But first, we need to prepare the data:

Data Preprocessing

Let's get all normal heartbeats and drop the target (class) column:

```
PYTHON
1  normal_df = df[df.target == str(CLASS_NORMAL)].drop(labels='target', axis=1)
2  normal_df.shape
```

```
1  (2919, 140)
```

We'll merge all other classes and mark them as anomalies:

```python
anomaly_df = df[df.target != str(CLASS_NORMAL)].drop(labels='target', axis=1)
anomaly_df.shape
```

```
(2081, 140)
```

We'll split the normal examples into train, validation and test sets:

```python
train_df, val_df = train_test_split(
  normal_df,
  test_size=0.15,
  random_state=RANDOM_SEED
)

val_df, test_df = train_test_split(
  val_df,
  test_size=0.33,
  random_state=RANDOM_SEED
)
```

We need to convert our examples into tensors, so we can use them to train our Autoencoder. Let's write a helper function for that:

```python
def create_dataset(df):

  sequences = df.astype(np.float32).to_numpy().tolist()

  dataset = [torch.tensor(s).unsqueeze(1).float() for s in sequences]

  n_seq, seq_len, n_features = torch.stack(dataset).shape

```

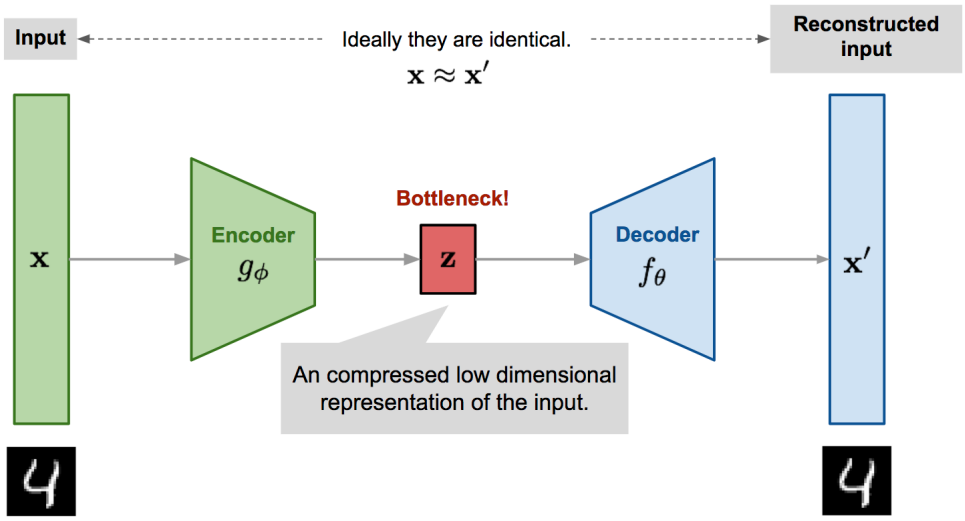
```
    return dataset, seq_len, n_features
```

Each Time Series will be converted to a 2D Tensor in the shape *sequence length* x *number of features* (140x1 in our case).

Let's create some datasets:

```python
train_dataset, seq_len, n_features = create_dataset(train_df)
val_dataset, _, _ = create_dataset(val_df)
test_normal_dataset, _, _ = create_dataset(test_df)
test_anomaly_dataset, _, _ = create_dataset(anomaly_df)
```

LSTM Autoencoder



Autoencoder

Sample Autoencoder Architecture Image Source

The general Autoencoder architecture consists of two components. An *Encoder* that compresses the input and a *Decoder* that tries to reconstruct it.

We'll use the LSTM Autoencoder from this GitHub repo with some small tweaks. Our model's job is to reconstruct Time Series data. Let's start with the *Encoder*:

```python
class Encoder(nn.Module):

  def __init__(self, seq_len, n_features, embedding_dim=64):
    super(Encoder, self).__init__()

    self.seq_len, self.n_features = seq_len, n_features
    self.embedding_dim, self.hidden_dim = embedding_dim, 2 * embedding_dim

    self.rnn1 = nn.LSTM(
      input_size=n_features,
      hidden_size=self.hidden_dim,
      num_layers=1,
      batch_first=True
    )

    self.rnn2 = nn.LSTM(
      input_size=self.hidden_dim,
      hidden_size=embedding_dim,
      num_layers=1,
      batch_first=True
    )

  def forward(self, x):
    x = x.reshape((1, self.seq_len, self.n_features))

    x, (_, _) = self.rnn1(x)
    x, (hidden_n, _) = self.rnn2(x)

    return hidden_n.reshape((self.n_features, self.embedding_dim))
```

The *Encoder* uses two LSTM layers to compress the Time Series data input.

Next, we'll decode the compressed representation using a *Decoder*:

PYTHON

```python
class Decoder(nn.Module):

  def __init__(self, seq_len, input_dim=64, n_features=1):
    super(Decoder, self).__init__()

    self.seq_len, self.input_dim = seq_len, input_dim
    self.hidden_dim, self.n_features = 2 * input_dim, n_features

    self.rnn1 = nn.LSTM(
      input_size=input_dim,
      hidden_size=input_dim,
      num_layers=1,
      batch_first=True
    )

    self.rnn2 = nn.LSTM(
      input_size=input_dim,
      hidden_size=self.hidden_dim,
      num_layers=1,
      batch_first=True
    )

    self.output_layer = nn.Linear(self.hidden_dim, n_features)

  def forward(self, x):
    x = x.repeat(self.seq_len, self.n_features)
    x = x.reshape((self.n_features, self.seq_len, self.input_dim))

    x, (hidden_n, cell_n) = self.rnn1(x)
    x, (hidden_n, cell_n) = self.rnn2(x)
    x = x.reshape((self.seq_len, self.hidden_dim))

```

```
33      return self.output_layer(x)
```

Our Decoder contains two LSTM layers and an output layer that gives the final reconstruction.

Time to wrap everything into an easy to use module:

PYTHON

```python
class RecurrentAutoencoder(nn.Module):

  def __init__(self, seq_len, n_features, embedding_dim=64):
    super(RecurrentAutoencoder, self).__init__()

    self.encoder = Encoder(seq_len, n_features, embedding_dim).to(device)
    self.decoder = Decoder(seq_len, embedding_dim, n_features).to(device)

  def forward(self, x):
    x = self.encoder(x)
    x = self.decoder(x)

    return x
```

Our Autoencoder passes the input through the Encoder and Decoder. Let's create an instance of it:

PYTHON

```python
model = RecurrentAutoencoder(seq_len, n_features, 128)
model = model.to(device)
```

Training

Let's write a helper function for our training process:

```python
def train_model(model, train_dataset, val_dataset, n_epochs):
  optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
  criterion = nn.L1Loss(reduction='sum').to(device)
  history = dict(train=[], val=[])

  best_model_wts = copy.deepcopy(model.state_dict())
  best_loss = 10000.0

  for epoch in range(1, n_epochs + 1):
    model = model.train()

    train_losses = []
    for seq_true in train_dataset:
      optimizer.zero_grad()

      seq_true = seq_true.to(device)
      seq_pred = model(seq_true)

      loss = criterion(seq_pred, seq_true)

      loss.backward()
      optimizer.step()

      train_losses.append(loss.item())

    val_losses = []
    model = model.eval()
    with torch.no_grad():
      for seq_true in val_dataset:

        seq_true = seq_true.to(device)
        seq_pred = model(seq_true)

```

```
        loss = criterion(seq_pred, seq_true)
        val_losses.append(loss.item())

    train_loss = np.mean(train_losses)
    val_loss = np.mean(val_losses)

    history['train'].append(train_loss)
    history['val'].append(val_loss)

    if val_loss < best_loss:
      best_loss = val_loss
      best_model_wts = copy.deepcopy(model.state_dict())

    print(f'Epoch {epoch}: train loss {train_loss} val loss {val_loss}')

  model.load_state_dict(best_model_wts)
  return model.eval(), history
```

At each epoch, the training process feeds our model with all training examples and evaluates the performance on the validation set. Note that we're using a batch size of 1 (our model sees only 1 sequence at a time). We also record the training and validation set losses during the process.

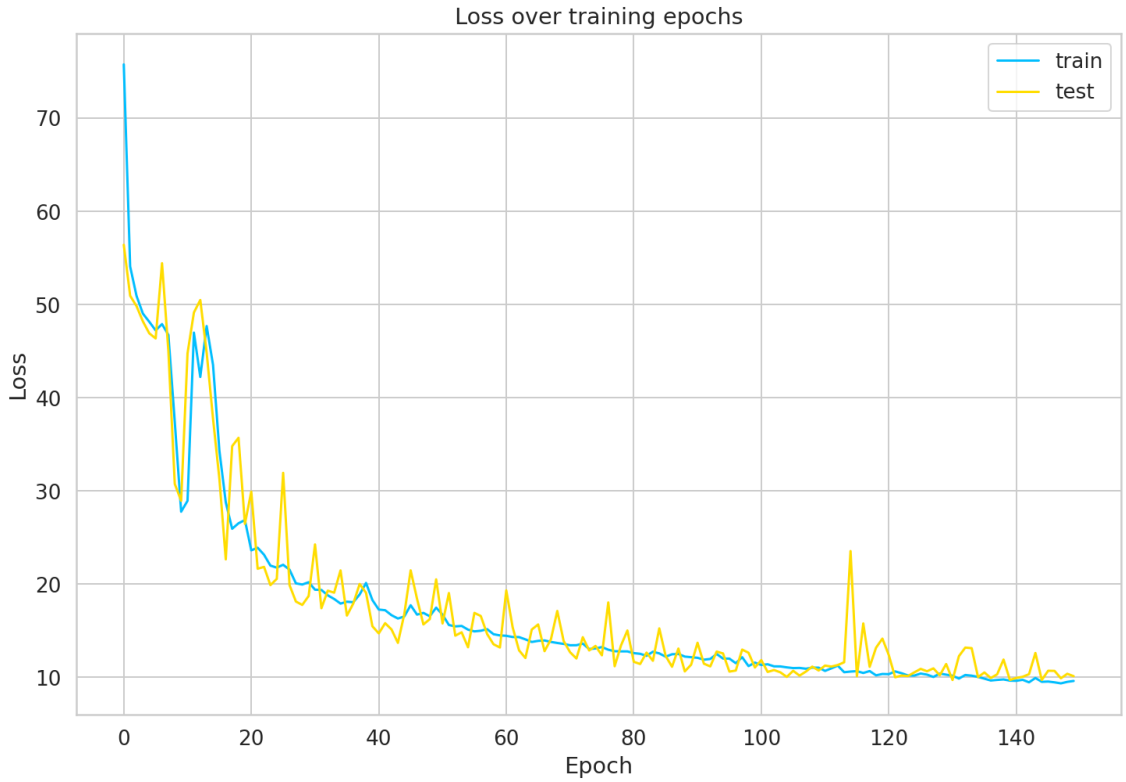
Note that we're minimizing the L1Loss, which measures the MAE (mean absolute error). Why? The reconstructions seem to be better than with MSE (mean squared error).

We'll get the version of the model with the smallest validation error. Let's do some training:

PYTHON

```
model, history = train_model(
  model,
  train_dataset,
```

```
    val_dataset,
    n_epochs=150
)
```



png

Our model converged quite well. Seems like we might've needed a larger validation set to smoothen the results, but that'll do for now.

Saving the model

Let's store the model for later use:

```python
MODEL_PATH = 'model.pth'

torch.save(model, MODEL_PATH)
```

Uncomment the next lines, if you want to download and load the pre-trained model:

```python
# !gdown --id 1jEYx5wGsb7Ix8cZAw3l5p5pOwHs3_I9A
# model = torch.load('model.pth')
# model = model.to(device)
```

Choosing a threshold

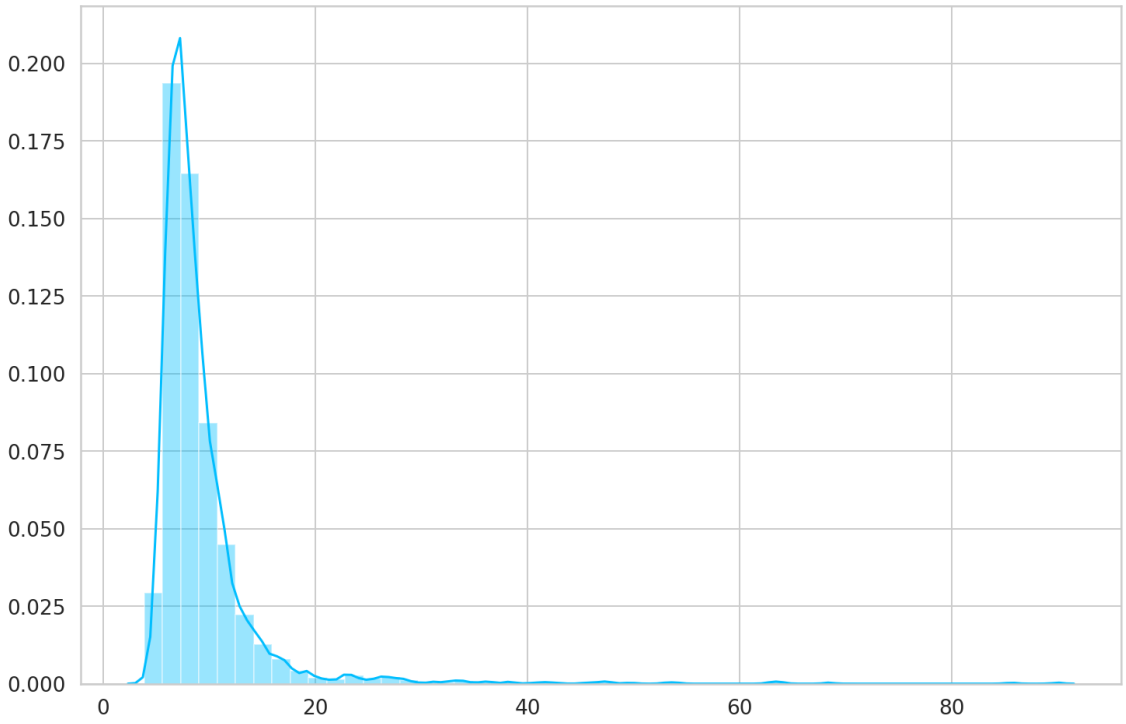
With our model at hand, we can have a look at the reconstruction error on the training set. Let's start by writing a helper function to get predictions from our model:

```python
def predict(model, dataset):
  predictions, losses = [], []
  criterion = nn.L1Loss(reduction='sum').to(device)
  with torch.no_grad():
    model = model.eval()
    for seq_true in dataset:
      seq_true = seq_true.to(device)
      seq_pred = model(seq_true)

      loss = criterion(seq_pred, seq_true)

      predictions.append(seq_pred.cpu().numpy().flatten())
      losses.append(loss.item())
  return predictions, losses
```

Our function goes through each example in the dataset and records the predictions and losses. Let's get the losses and have a look at them:

```python
_, losses = predict(model, train_dataset)
```

```
2
3  sns.distplot(losses, bins=50, kde=True);
```



png

```
PYTHON
1  THRESHOLD = 26
```

Evaluation

Using the threshold, we can turn the problem into a simple binary classification task:

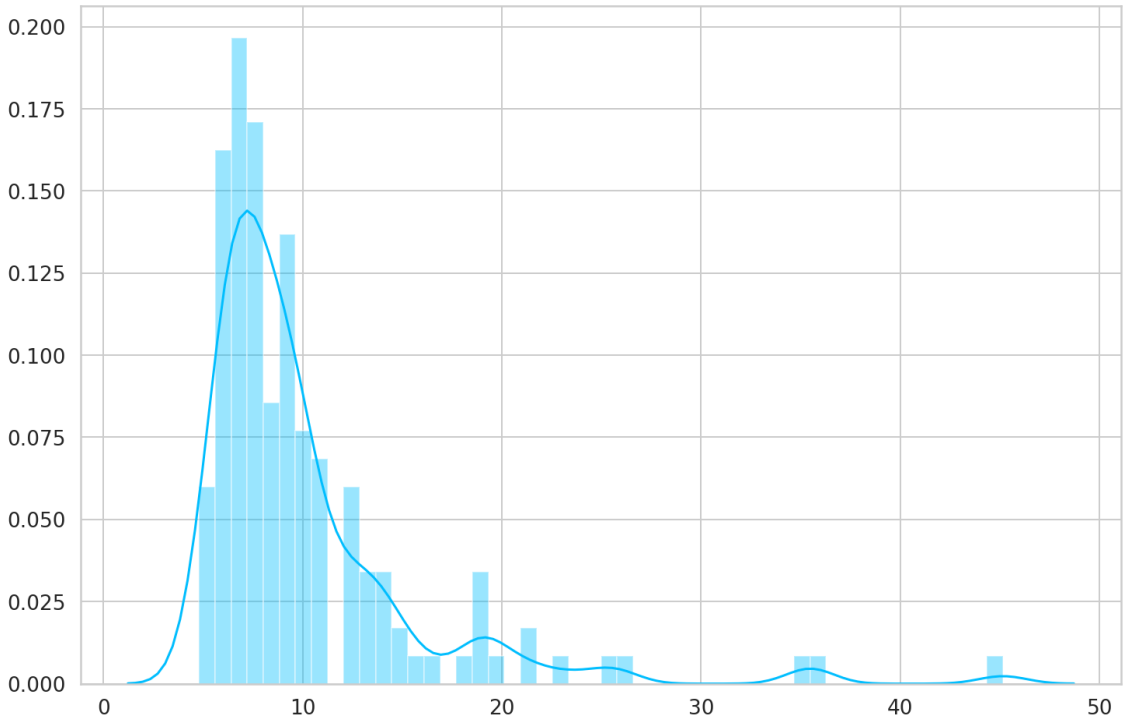
- If the reconstruction loss for an example is below the threshold, we'll classify it as a *normal* heartbeat
- Alternatively, if the loss is higher than the threshold, we'll classify it as an anomaly

Normal hearbeats

Let's check how well our model does on normal heartbeats. We'll use the normal

heartbeats from the test set (our model haven't seen those):

```python
predictions, pred_losses = predict(model, test_normal_dataset)
sns.distplot(pred_losses, bins=50, kde=True);
```



png

We'll count the correct predictions:

```python
correct = sum(l <= THRESHOLD for l in pred_losses)
print(f'Correct normal predictions: {correct}/{len(test_normal_dataset)}')
```

```
Correct normal predictions: 142/145
```

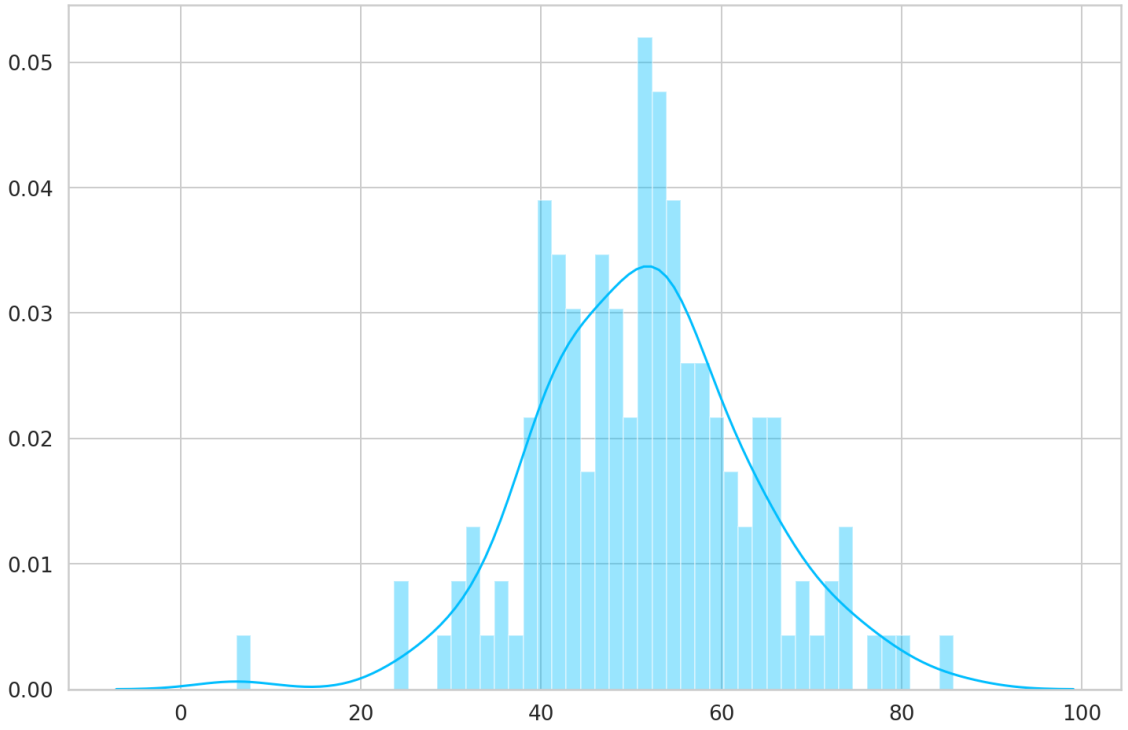
Anomalies

We'll do the same with the anomaly examples, but their number is much higher. We'll get a subset that has the same size as the normal heartbeats:

```python
anomaly_dataset = test_anomaly_dataset[:len(test_normal_dataset)]
```

Now we can take the predictions of our model for the subset of anomalies:

```python
predictions, pred_losses = predict(model, anomaly_dataset)
sns.distplot(pred_losses, bins=50, kde=True);
```



png

Finally, we can count the number of examples above the threshold (considered as anomalies):

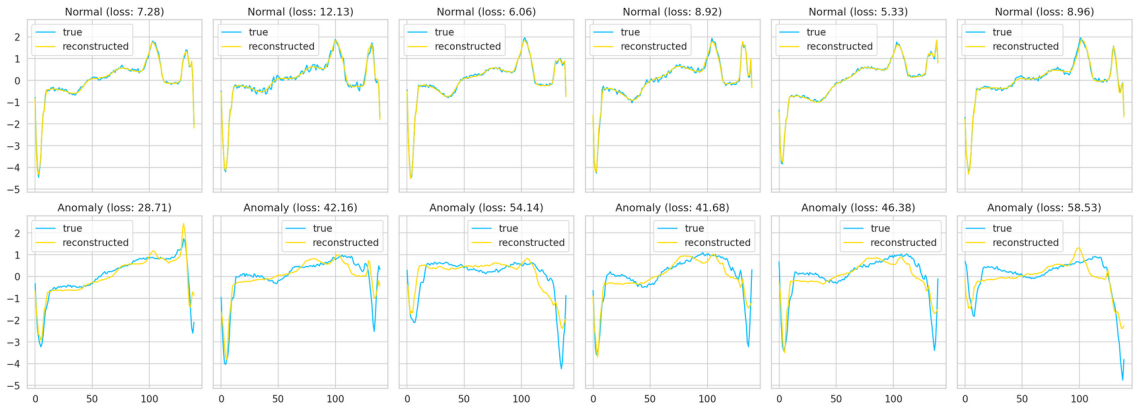
```python
correct = sum(l > THRESHOLD for l in pred_losses)
print(f'Correct anomaly predictions: {correct}/{len(anomaly_dataset)}')
```

```
Correct anomaly predictions: 142/145
```

We have very good results. In the real world, you can tweak the threshold depending on what kind of errors you want to tolerate. In this case, you might want to have more false positives (normal heartbeats considered as anomalies) than false negatives (anomalies considered as normal).

Looking at Examples

We can overlay the real and reconstructed Time Series values to see how close they are. We'll do it for some normal and anomaly cases:



png

Summary

In this tutorial, you learned how to create an LSTM Autoencoder with PyTorch and use it to detect heartbeat anomalies in ECG data.

- Run the complete notebook in your browser (Google Colab)
- Read the Getting Things Done with Pytorch book

You learned how to:

- Prepare a dataset for Anomaly Detection from Time Series Data
- Build an LSTM Autoencoder with PyTorch
- Train and evaluate your model
- Choose a threshold for anomaly detection
- Classify unseen examples as normal or anomaly

While our Time Series data is univariate (we have only 1 feature), the code should work for multivariate datasets (multiple features) with little or no modification. Feel free to try it!

References

- Sequitur - Recurrent Autoencoder (RAE)
- Towards Never-Ending Learning from Time Series Streams
- LSTM Autoencoder for Anomaly Detection

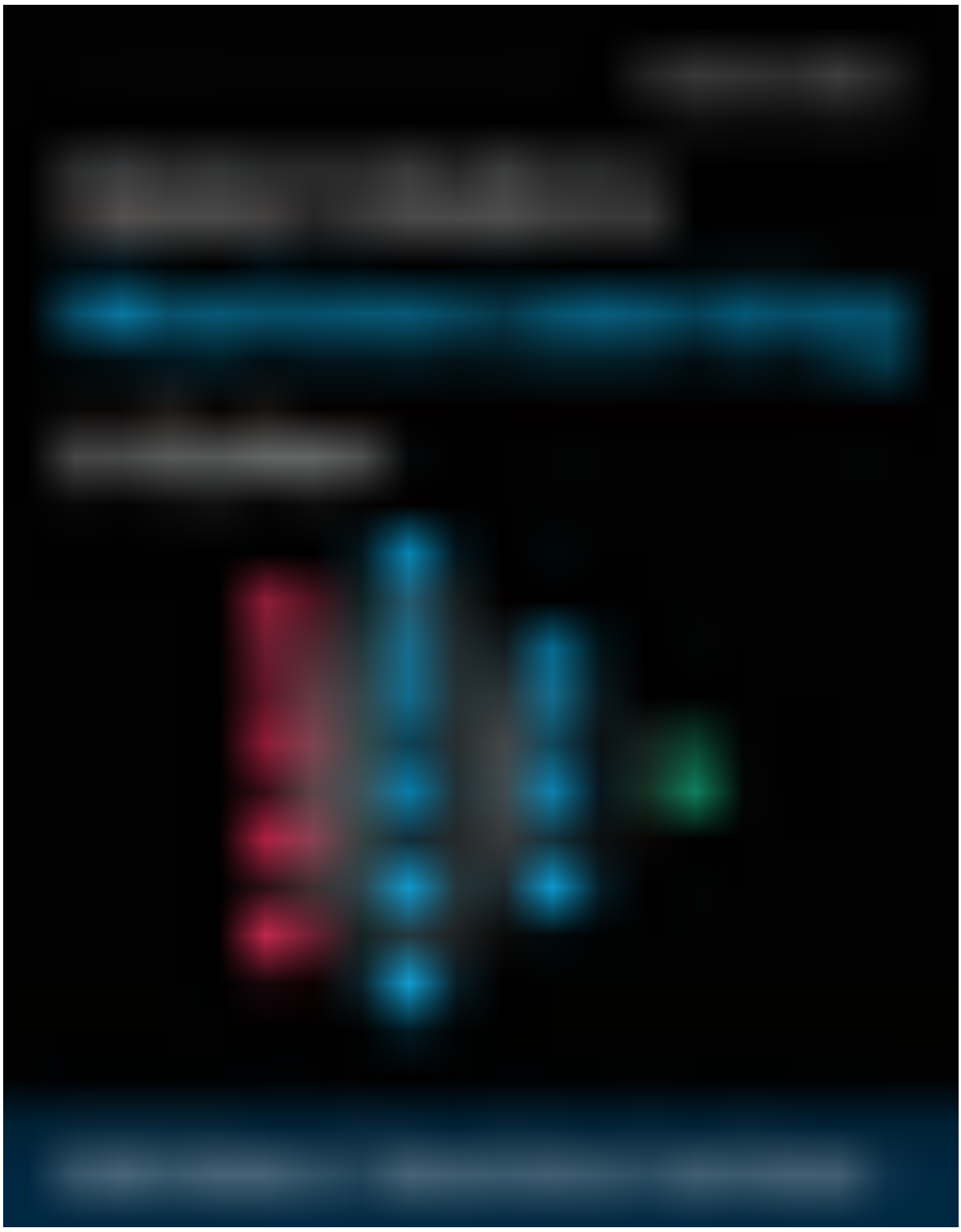
Hacker's Guide to Neural Networks in JavaScript

Build Machine Learning models (especially Deep Neural Networks) that you can easily integrate with existing or new web apps. Think of your ReactJs, Vue,

or Angular app enhanced with the power of Machine Learning models.

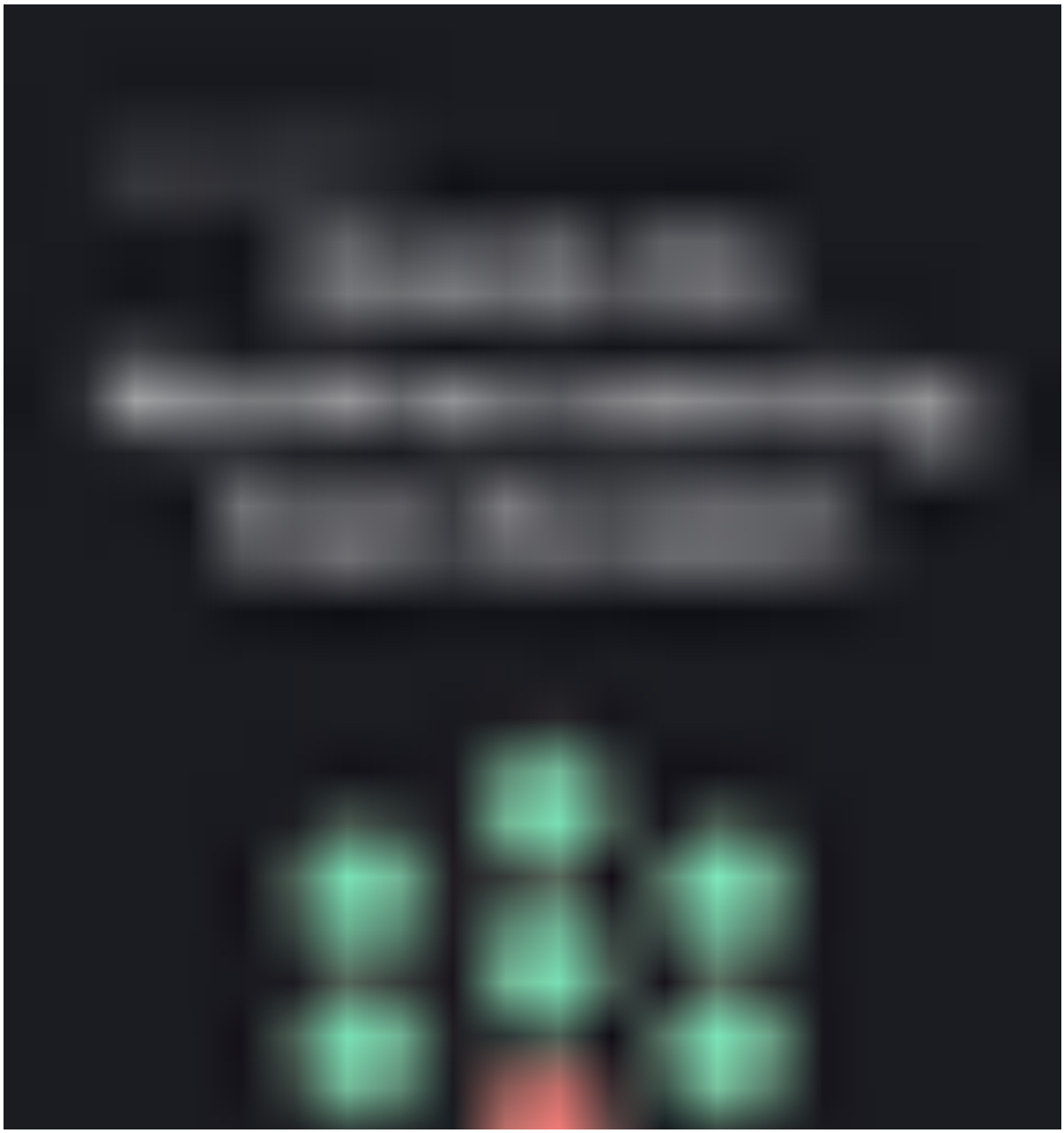
Get SH*T Done with PyTorch

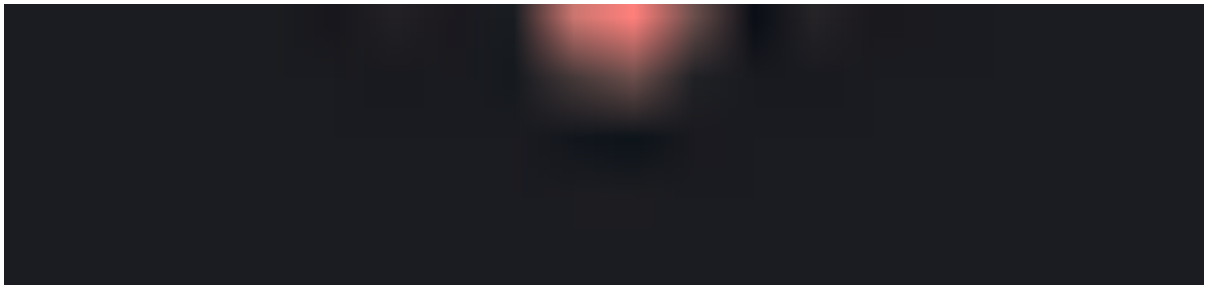
Learn how to solve real-world problems with Deep Learning models (NLP, Computer Vision, and Time Series). Go from prototyping to deployment with PyTorch and Python!



Hacker's Guide to Machine Learning with Python

This book brings the fundamentals of Machine Learning to you, using tools and techniques used to solve real-world problems in Computer Vision, Natural Language Processing, and Time Series analysis. The skills taught in this book will lay the foundation for you to advance your journey to Machine Learning Mastery!





Hands-On Machine Learning from Scratch

This book will guide you on your journey to deeper Machine Learning understanding by developing algorithms in Python from scratch! Learn why and when Machine learning is the right tool for the job and how to improve low performing models!

What do you think?

8 Responses

 Upvote

 Funny

 Love

 Surprised

 Angry

 Sad

17 Comments 1 Login ▼

G

Join the discussion…

LOG IN WITH

OR SIGN UP WITH DISQUS ?

Name

♡ • Share

Best Newest Oldest